

# Component & Protocol Reference

Detailed reference for every class in `DigitalHumanRuntimeSDK`: what each component does, every tunable property, the WebSocket protocol it speaks, and how to wire it into a MetaHuman's Animation Blueprints. See the [top-level README](#) for the architecture overview and known limitations.

## Requirements

- Unreal Engine 5.6, Win64
- A MetaHuman character already placed in the project
- A backend that speaks the protocol described below over a WebSocket connection

## Installation

1. Copy the `DigitalHumanRuntimeSDK` folder into `<Project>/Plugins/`.
2. Right-click the project's `.uproject` file → **Generate Visual Studio project files.**
3. Build the generated solution (Development Editor / Win64), or open the project and let the editor prompt a rebuild.
4. In the editor, go to **Edit → Plugins**, search "Digital Human", and confirm it's enabled.

## Components

### `UDigitalHumanSubsystem`

`UGameInstanceSubsystem` — the single owner of the backend WebSocket connection for the whole game instance. Every other component talks to the backend through this subsystem's events rather than opening its own connection, so multiple digital-human components in a level correctly share one connection. Access it from Blueprint via **Get Game Instance Subsystem** → class `DigitalHumanSubsystem`.

| Function                         | Signature                                                  | Notes                                                                               |
|----------------------------------|------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <code>Connect</code>             | <code>(FString ServerURL)</code>                           | e.g. <code>ws://127.0.0.1:8765</code>                                               |
| <code>Disconnect</code>          | <code>()</code>                                            | Also disables auto-reconnect until <code>Connect</code> is called again             |
| <code>IsConnected</code>         | <code>() -&gt; bool</code>                                 |                                                                                     |
| <code>SendJSON</code>            | <code>(FString JsonText)</code>                            | Sent as a text frame; caller is responsible for valid JSON                          |
| <code>SendBinary</code>          | <code>(TArray&lt;uint8&gt; Bytes)</code>                   | Sent as a binary frame                                                              |
| <code>SetAutoReconnect</code>    | <code>(bool bEnabled)</code>                               | Enabled by default                                                                  |
| <code>SetHeartbeatEnabled</code> | <code>(bool bEnabled, float IntervalSeconds = 15.0)</code> | Sends <code>{"type":"heartbeat"}</code> text frames on the interval while connected |

| Event                          | Payload                                                                       |
|--------------------------------|-------------------------------------------------------------------------------|
| <code>OnConnected</code>       | —                                                                             |
| <code>OnConnectionError</code> | <code>FString ErrorMessage</code>                                             |
| <code>OnDisconnected</code>    | <code>int32 StatusCode, FString Reason</code>                                 |
| <code>OnJsonReceived</code>    | <code>FString JsonText</code> — every text frame, unparsed                    |
| <code>OnBinaryReceived</code>  | <code>TArray&lt;uint8&gt; Bytes</code> — every binary frame, e.g. a PCM chunk |

Internally backed by `FDigitalHumanWebSocketManager`, a plain (non-`UObject`) class not exposed to Blueprint. Reconnection uses exponential backoff: 1.0s initial delay, doubling on each attempt, capped at a 30.0s ceiling.

### `UDigitalHumanAudioPlayerComponent`

Streams backend-provided PCM audio to a speaker in real time via `USoundWaveProcedural` — no temp files, no wait for a complete file. By default it auto-subscribes to the subsystem's `OnJsonReceived` / `OnBinaryReceived` events on `BeginPlay`; no manual wiring needed for the backend-driven path.

| Property                             | Default           | Description                                               |
|--------------------------------------|-------------------|-----------------------------------------------------------|
| <code>AudioVolume</code>             | <code>1.0</code>  | Applied to the internal Audio Component                   |
| <code>bAutoSubscribeToBackend</code> | <code>true</code> | Set false to drive playback only via <code>FeedPCM</code> |

| Function                                                | Notes                                                                          |
|---------------------------------------------------------|--------------------------------------------------------------------------------|
| <code>FeedPCM(PCMBytes, SampleRate, NumChannels)</code> | Queues a PCM16 chunk for playback; safe to call every time a new chunk arrives |
| <code>StopAndFlush()</code>                             | Stops immediately and clears queued-but-unplayed audio                         |
| <code>IsPlaying() -&gt; bool</code>                     |                                                                                |

| Event                            | Payload                                                                                                                                                                      |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>OnPCMChunkProcessed</code> | <code>PCMBytes, SampleRate, NumChannels</code> — fires once a chunk is queued; downstream systems (the facial driver) subscribe here instead of re-parsing the network layer |

Resampling is not hand-rolled: `USoundWaveProcedural` is configured with the backend-declared sample rate/channel count, and the engine's own audio mixer resamples to the output device automatically.

`UDigitalHumanFacialDriverComponent`

Extracts real-time facial animation values directly from raw PCM — no phoneme data, no ML model. By default it finds a `UDigitalHumanAudioPlayerComponent` on the same Actor and subscribes to its `OnPCMChunkProcessed` automatically.

Base signals

| Property                            | Default           | Description                                                                                                                  |
|-------------------------------------|-------------------|------------------------------------------------------------------------------------------------------------------------------|
| <code>EnvelopeSmoothingSpeed</code> | <code>12.0</code> | How fast <code>MouthOpenAmount</code> / <code>MouthWideAmount</code> chase their target. Higher = snappier, lower = smoother |
| <code>MouthOpenGain</code>          | <code>4.0</code>  | Gain on raw RMS before clamping to 0-1                                                                                       |
| <code>MouthWideGain</code>          | <code>6.0</code>  | Gain on raw zero-crossing rate before clamping to 0-1                                                                        |
| <code>SilenceTimeoutSeconds</code>  | <code>0.3</code>  | No chunk for longer than this and the mouth eases back to 0                                                                  |

| Output                       | Range | Source             |
|------------------------------|-------|--------------------|
| <code>MouthOpenAmount</code> | 0-1   | RMS volume level   |
| <code>MouthWideAmount</code> | 0-1   | Zero-crossing rate |

Event `OnFaceCurvesUpdated(MouthOpenAmount, MouthWideAmount)` fires on meaningful change, as an alternative to polling every tick.

**Multi-band extension** — three RBJ-cookbook band-pass biquads (Direct Form II Transposed, stateful across chunks) split the signal into low/mid/high energy bands. This is frequency-content heuristics, not phoneme classification.

| Property                              | Default            | Description                                                                                |
|---------------------------------------|--------------------|--------------------------------------------------------------------------------------------|
| <code>bEnableMultiBandAnalysis</code> | <code>true</code>  | Master switch — off falls back to the base two-signal behavior with zero interference risk |
| <code>MouthFunnelGain</code>          | <code>3.0</code>   | Gain on low-band energy dominance                                                          |
| <code>FricativeGain</code>            | <code>4.0</code>   | Gain on high-band noise-like energy                                                        |
| <code>PlosiveOnsetThreshold</code>    | <code>0.08</code>  | RMS jump (chunk vs. slow moving average) required to register a plosive onset              |
| <code>PlosiveSilenceThreshold</code>  | <code>0.025</code> | The moving average must be below this immediately before the onset                         |
| <code>PlosiveAttackDuration</code>    | <code>0.02</code>  | Seconds                                                                                    |
| <code>PlosiveDecayDuration</code>     | <code>0.12</code>  | Seconds                                                                                    |

| Output                         | Range | Approximates                                                                                                |
|--------------------------------|-------|-------------------------------------------------------------------------------------------------------------|
| <code>MouthFunnelAmount</code> | 0-1   | Rounded-lip shape ("oo"/"oh")                                                                               |
| <code>FricativeAmount</code>   | 0-1   | Teeth/lip tension ("s"/"f"/"th" — not distinguished from each other)                                        |
| <code>PlosiveAmount</code>     | 0-1   | A short attack/decay pulse approximating a stop consonant (b/p/t/d/k/g — not distinguished from each other) |

`AnalyzePCMChunk(PCMBytes, SampleRate, NumChannels)` is exposed for manually feeding chunks when auto-subscribe is disabled. See the top-level README's Known Limitations for how the plosive detector behaves at real chunk granularity.

UDigitalHumanIdleAnimationComponent

Procedural "alive" motion that runs continuously and independently of network/audio state — nothing here touches MouthOpenAmount / MouthWideAmount . Layer it underneath the audio-driven curves in the same AnimBP.

| Group        | Property                                                   | Default            |
|--------------|------------------------------------------------------------|--------------------|
| Blink        | bEnableBlinking                                            | true               |
| Blink        | BlinkIntervalMinSeconds / MaxSeconds                       | 2.0 / 6.0          |
| Blink        | BlinkCloseDuration / BlinkHoldDuration / BlinkOpenDuration | 0.08 / 0.03 / 0.12 |
| Breathing    | bEnableBreathing                                           | true               |
| Breathing    | BreathingCycleSeconds                                      | 4.0                |
| Eye saccades | bEnableEyeSaccades                                         | true               |
| Eye saccades | SaccadeIntervalMinSeconds / MaxSeconds                     | 1.0 / 4.0          |
| Eye saccades | SaccadeMaxOffsetDegrees                                    | 3.0                |
| Eye saccades | SaccadeMoveDuration                                        | 0.05               |
| Head         | bEnableHeadMicroMovement                                   | true               |
| Head         | HeadMicroMovementAmplitudeDegrees                          | 1.5                |
| Head         | HeadMicroMovementSpeed                                     | 0.3                |

| Output                  | Type      | Notes                                                                                                           |
|-------------------------|-----------|-----------------------------------------------------------------------------------------------------------------|
| BlinkAmount             | float 0-1 | 0 = open, 1 = closed                                                                                            |
| BreathingAmount         | float 0-1 | Smooth sine cycle                                                                                               |
| EyeSaccadeOffset        | FVector2D | Small Yaw/Pitch offset in degrees — add to a Look-At target's aim, don't use as an absolute rotation            |
| HeadMicroMovementOffset | FRotator  | Perlin-noise-driven (not uniform random, so it reads as organic); additive, feed into a Transform (Modify) Bone |

TriggerBlink() forces a blink immediately, ignoring the random timer.

UDigitalHumanBehaviorComponent

A conversation-state machine ( Idle / Listening / Thinking / Speaking ) that ties the other components together.

| Property                       | Default | Description                                                                                                                                                          |
|--------------------------------|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| bAutoDetectSpeakingFromAudio   | true    | Treats AudioPlayerComponent::IsPlaying() on the same Actor as authoritative for Speaking                                                                             |
| bAutoSubscribeToBackend        | true    | Listens for {"type": "state", ...} from the backend                                                                                                                  |
| bSuppressSaccadesWhileSpeaking | true    | Finds DigitalHumanIdleAnimationComponent on the same Actor and suppresses saccades while Speaking, restoring the prior setting (not force-enabling) once speech ends |

| Function                  | Notes                                                                                                                                                                                                                  |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SetState(NewState)        | Ignored (logged) if real audio is playing and the requested state isn't Speaking — audio always wins, since forcing the state away would visibly contradict the moving mouth. Re-entering the current state is a no-op |
| SetLookAtTarget(Actor)    | Stores a reference only — wiring it into actual eye rotation is scene-specific and left to the caller                                                                                                                  |
| ShouldAvertGaze() -> bool | True while Thinking — a convenience flag for a "recalling information" gaze-away cue                                                                                                                                   |

CurrentState and CurrentLookAtTarget are read-only. OnStateChanged(OldState, NewState) fires on every transition. Once playback ends, state falls back to Idle automatically.

UDigitalHumanEmotionComponent

Smoothly blends toward named expression-intensity targets instead of hard-switching curves. Independent of the Behavior state machine — nothing reads conversation state automatically; wire OnStateChanged → SetEmotion() yourself if that coupling is wanted.

| Property                | Default | Description                             |
|-------------------------|---------|-----------------------------------------|
| TransitionSpeed         | 3.0     | Higher = faster blend on emotion change |
| bAutoSubscribeToBackend | true    | Listens for {"type":"emotion",...}      |

Emotions: Neutral, Happy, Friendly, Concerned, Excited.

Outputs (all float, 0-1, read-only): SmileAmount, BrowRaiseAmount, EyeSquintAmount. SetEmotion(NewEmotion) sets the target; OnEmotionChanged(OldEmotion, NewEmotion) fires on change. Exact expression curve names are rig-specific and must be confirmed in the Skeleton Editor before wiring.

UDigitalHumanGestureComponent

Two independent pieces:

- Montage-based gestures** — a GestureLibrary ( FName -> UAnimMontage\* ) populated by the project. The plugin ships zero gesture animations. BodyMeshComponent must be set explicitly to the Actor's Body skeletal mesh (not Face — a MetaHuman has two, and the component can't safely guess). PlayGesture(GestureName) -> bool plays the named montage; returns false and logs a warning (never crashes) on a missing name or unset mesh. Backend-triggerable via {"type":"gesture","gesture":"<name>"} when bAutoSubscribeToBackend is true.
- Procedural head nod** — needs no external assets. NodAmplitudeDegrees ( 8.0 ), NodDownDuration ( 0.18 ), NodUpDuration ( 0.22 ). TriggerNod() starts a nod; safe to call again mid-nod, it retriggers from the current position. Outputs NodPitchOffset (additive Pitch, degrees) — combine with HeadMicroMovementOffset, don't replace it.

## WebSocket Protocol

All messages are JSON text frames unless noted otherwise. The backend is responsible for framing; the plugin does not assume any particular message ordering beyond "a format header precedes the binary frames it describes."

| Direction        | Message                                                                                          | Consumed by                                                                 |
|------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Backend → plugin | {"type":"audio_start","sampleRate":24000,"channels":1} (or {"type":"audio" with the same fields) | AudioPlayerComponent — declares the format for binary frames that follow    |
| Backend → plugin | binary frame                                                                                     | AudioPlayerComponent — raw PCM16 bytes in the most recently declared format |
| Backend → plugin | {"type":"audio_end"}                                                                             | AudioPlayerComponent — informational only                                   |
| Backend → plugin | {"type":"interrupt"}                                                                             | AudioPlayerComponent — stops playback and flushes queued audio immediately  |
| Backend → plugin | {"type":"state","state":"idle" "listening" "thinking" "speaking"}                                | BehaviorComponent                                                           |
| Backend → plugin | {"type":"emotion","emotion":"neutral" "happy" "friendly" "concerned" "excited"}                  | EmotionComponent                                                            |
| Backend → plugin | {"type":"gesture","gesture":"<name>"}                                                            | GestureComponent — <name> must match a GestureLibrary key                   |
| Plugin → backend | {"type":"heartbeat"}                                                                             | Optional, sent on SetHeartbeatEnabled(true, IntervalSeconds) 's interval    |
| Plugin → backend | anything via SendJSON / SendBinary                                                               | Application-defined                                                         |

## AnimBP Wiring Guide

### Face curves ( Face\_PostProcess\_AnimBP )

- Add DigitalHumanAudioPlayerComponent and DigitalHumanFacialDriverComponent (and, if used, DigitalHumanIdleAnimationComponent / DigitalHumanEmotionComponent) to the MetaHuman Actor.
- In **Event Blueprint Initialize Animation**, cache each component reference once: **Get Owing Actor → Get Component By Class** (the relevant class) → store in a local variable. Avoid calling **Get Component By Class** every tick.
- In the Anim Graph, on the relevant **Modify Curve** node, right-click → **Add Curve Pin** for each rig curve to drive. Curve names are rig-specific — confirm the exact names in the Skeleton Editor's curve list before wiring; do not assume a name.
- Wire each new pin from the matching **Get <Property>** call on the cached component reference (e.g. **Get Mouth Open Amount**, **Get Blink Amount**, **Get Smile Amount**).
- Drive this every frame from **Event Blueprint Update Animation**.

Head bone ( `ABP_BodyIdleMotion` or equivalent)

Epic's `ABP_Face_PostProcess` runs after `Face_PostProcess_AnimBP` and copies all Face-component bone transforms — including the head — from the Body component's pose every frame. Head-bone edits therefore have to go in a separate AnimBP driving the **Body** component, not in `Face_PostProcess_AnimBP`.

- 1. Add `DigitalHumanIdleAnimationComponent` (and, if used, `DigitalHumanGestureComponent`) to the Body Actor.
- 2. Cache the component reference(s) once in **Event Blueprint Initialize Animation**.
- 3. In the Anim Graph, add a **Transform (Modify) Bone** node targeting the head bone, **Local Space**, **Additive** rotation mode, placed after any existing head-tracking IK so it adds on top rather than fighting it.
- 4. To combine idle drift with the procedural nod: **Break Rotator** both `HeadMicroMovementOffset` and `NodPitchOffset`, add the two Pitch values, **Make Rotator** with the combined Pitch and the idle offset's Yaw/Roll untouched, and feed that into the Transform (Modify) Bone's Rotation input.

Tuning Reference

| Component     | Property                                                                                     | Default                                                  | Effect                                                                                                                                                          |
|---------------|----------------------------------------------------------------------------------------------|----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FacialDriver  | <code>EnvelopeSmoothingSpeed</code>                                                          | <code>12.0</code>                                        | Higher = snappier mouth response, lower = smoother/laggier                                                                                                      |
| FacialDriver  | <code>MouthOpenGain</code>                                                                   | <code>4.0</code>                                         | Raise if the mouth barely opens; lower if it's pinned fully open — raw RMS loudness varies significantly by TTS voice                                           |
| FacialDriver  | <code>MouthWideGain</code>                                                                   | <code>6.0</code>                                         | Same idea, for mouth width vs. zero-crossing rate                                                                                                               |
| FacialDriver  | <code>PlosiveOnsetThreshold</code> / <code>PlosiveSilenceThreshold</code>                    | <code>0.08</code> / <code>0.025</code>                   | Dual-condition gate for the plosive pulse — see the top-level README's Known Limitations for its real-world firing behavior at sentence-level chunk granularity |
| IdleAnimation | <code>BlinkIntervalMinSeconds</code> / <code>MaxSeconds</code>                               | <code>2.0</code> / <code>6.0</code>                      | Blink frequency                                                                                                                                                 |
| IdleAnimation | <code>HeadMicroMovementAmplitudeDegrees</code> / <code>HeadMicroMovementSpeed</code>         | <code>1.5</code> / <code>0.3</code>                      | Keep under roughly 2° / 0.5 to avoid a "nervous tic" look                                                                                                       |
| Behavior      | <code>bSuppressSaccadesWhileSpeaking</code>                                                  | <code>true</code>                                        | Disable if saccades should keep running during speech                                                                                                           |
| Emotion       | <code>TransitionSpeed</code>                                                                 | <code>3.0</code>                                         | Higher = faster expression blend on an emotion change                                                                                                           |
| Gesture       | <code>NodAmplitudeDegrees</code> / <code>NodDownDuration</code> / <code>NodUpDuration</code> | <code>8.0</code> / <code>0.18</code> / <code>0.22</code> | Procedural nod shape                                                                                                                                            |

Troubleshooting

Plugin doesn't appear in Edit → Plugins, or won't enable

- Confirm the folder sits at `<Project>/Plugins/DigitalHumanRuntimeSDK/` — same level as the project's `Source` folder, not nested inside another folder.
- Regenerate project files and rebuild.

Compile errors after adding the plugin

- Confirm the project builds against UE 5.6, matching `EngineVersion` in `DigitalHumanRuntimeSDK.uplugin`.
- Delete the plugin's and the project's `Binaries` / `Intermediate` folders, then regenerate project files — this clears a stale incremental build, the most common cause of an otherwise-inexplicable module error.
- Confirm the "Game development with C++" workload is installed if `UnrealBuildTool` reports a missing toolchain.

No audio plays

- Confirm `UDigitalHumanSubsystem::Connect()` was called with a reachable `ws://` URL and `IsConnected()` returns `true`.
- Confirm the backend sends an `audio_start` / `audio` JSON header before any binary PCM frames — frames received before a format header are dropped.
- Check the Output Log filtered to `LogDigitalHuman` for connection/format warnings.

Mouth doesn't move even though audio is playing

- Confirm `DigitalHumanFacialDriverComponent` and `DigitalHumanAudioPlayerComponent` are on the same Actor — auto-subscribe depends on this.
- Confirm the Modify Curve pins in `Face_PostProcess_AnimBP` are wired to the driver's outputs, not left on a placeholder source.
- If the mouth barely opens or stays pinned fully open, adjust `MouthOpenGain`.

Character doesn't blink or show idle head movement

- Confirm `DigitalHumanIdleAnimationComponent` is present and its curve/bone wiring matches the mouth-curve pattern.

- This component runs independently of any backend connection — if it isn't animating with zero network activity, the AnimBP wiring is the first thing to check, not the connection.

#### **Reconnect doesn't happen after a dropped connection**

- Confirm `SetAutoReconnect(false)` hasn't been called — it's enabled by default.
- Reconnect uses exponential backoff (1s → 30s ceiling); a gap of a few seconds before a new "Connected" log line is expected, not a bug.